

00 — CRAM SHEET (last-hour read)

Course: Software Quality and Testing (SQAT) — CSC 4133 · **Exam:** Mid, Ch 1–6 **Textbooks:** Naik & Tripathy · Jeff Tian

File-name note: the .pptx file names are shifted +1 from the chapter inside. Ch.02 file = **Chapter 1**, Ch.03 file = **Chapter 2**, ... Ch.06 Unit Testing file = **Chapter 6**. This pack uses the *content* chapter numbers (what the exam uses).

CH 1 — Basics of Software Quality Assurance

- **Software** = programs + procedures + documentation + data. Two types: **Generic (Buy → COTS)**, **Customized (Build → bespoke)**.
- **Quality (ISO/IEC 9126):** totality of functionality + features that satisfy **stated or implied needs**.
- **Quality (IEEE 610):** degree to which a component/system/process meets specified requirements + user needs.
- **3 project challenges: Time, Cost, Scope.**
- **Famous defects: Therac-25** (radiation machine, AECL 1986, deaths from overdose — software bug) · **Ariane 5** (4 June 1996, exploded 37 s after launch — most expensive computer bug).

	SQA	SQC
Aim	Prevent defects	Identify & resolve defects
Program run?	Does NOT execute program	Always executes program
Who	All team members	Only testing team
Maps to	Verification (doing things right, process-oriented)	Validation (results as expected, product-oriented)
System type	Failure prevention system	Failure detection system
Scope	Full SDLC	Software testing life cycle

- **SQC review activities:** Requirement, Design, Code, Deployment Plan, Test Plan, Test Cases review.
- **Static analysis (manual):** examine code/docs, reason over all behaviours → code review, inspection, algorithm analysis. Slow.
- **Dynamic analysis (automated):** actually **execute** program to expose failures. → complementary to static.
- **Error → Fault → Failure → Defect** (memorize cold):
- **Error** = human action producing an incorrect result (injects a fault).
- **Fault** = incorrect step/process/data definition in the program (the underlying condition).
- **Failure** = inability of system to perform required function.
- **Defect** = behavioural deviation from requirement/spec; collectively = **"bugs"**.
- **SqE 3 stages: Planning → Execution → Decision.**
- **Complete/Exhaustive testing** ≈ impossible: input domain too large; too complex; can't build all environments.

CH 2 — Software Quality

- **Perspectives: External/Consumer** (customers, users) = **black box** · **Internal/Producer** (developers, testers, managers) = **white/clear box** · **Other** (3rd party).
- **5 Views of quality (Kitchenham & Pfleeger)** — mnemonic **"My Uncle Makes Pretty Vases"**:

#	View	One line
1	Mystical	Quality recognized by experience, not defined; good quality "stands out"
2	User	Meets user needs/expectations; good if satisfies many users
3	Manufacturing	Conformance to process/requirements; "right the first time" → CMM & ISO 9001
4	Product	Good internal properties → good external properties
5	Value-based	Willingness to pay; excellence (quality) + worth (value) ; cost/quality trade-off

- **Why measure quality:** Baseline · Quality improvement (based on cost) · Know present level for future planning.
- **Quality FACTOR** = behavioural characteristic of a system (external). Examples: **Correctness, Reliability, Efficiency, Performance, Integrity/Security, Usability**.
- **Quality CRITERION** = attribute of a factor, related to development (internal). Examples: **Modularity, Testability, Maintainability, Reusability, Flexibility, Portability, Interoperability**.
- **Model: McCall's** Quality Factors & Criteria.
- **Reliability** = probability software executes **without failure for a specific period of time**.
- **ISO-9126** — most influential; hierarchical (characteristics + non-overlapping sub-characteristics). **6 top-level characteristics** — mnemonic "**Fat Rabbits Usually Eat More Peas**":

Functionality	Reliability	Usability	Efficiency	Maintainability	Portability
what is needed	functions correctly	effort to use	resources needed	correct/improve/adapt	env → env

TRAP: Security is **NOT** an ISO-9126 top-level characteristic. Reliability is a **factor**; Modularity is a **criterion**.

CH 3 — Maturity Models

- **Software process** = set of activities to develop products. Benefits: **repeatable, measurable, improvable**.
- **Process tasks:** Requirements Analysis, Design/Modeling, Coding, Testing, Implementation/Integration, Operation/Maintenance, Documentation.
- **Why improve a process: Quality, Lead Time, Cost.**
- **3 models: CMM** = evaluate **development** process · **TPI** = **improve** testing process · **TMM** = **evaluate** testing process.

CMM — 5 levels (mnemonic "**I Really Don't Manage Overtime**"):

L	Name	Meaning
1	Initial	Disorganized/chaotic; depends on individuals; not repeatable
2	Repeatable	Basic project management; successes repeatable
3	Defined	Own standard process; documentation, standardization, integration
4	Managed	Monitors & controls via data collection & analysis
5	Optimizing	Constant improvement via feedback + innovation

TRAP: Level 4 = **Managed**, Level 5 = **Optimizing** (not reversed).

TMM — 5 levels (Ilene **Burnstein**) (mnemonic "**I Prefer Increasingly Mature Orgs**"):

L	Name	Key idea
1	Initial	No goals; testing after code; ad hoc; not tracked
2	Phase Definition	Testing+debugging goals; initiate test planning
3	Integration	Establish test group ; training; integrate testing into lifecycle
4	Management and Measurement	Org-wide review; test management; evaluate quality
5	Optimization / Defect Prevention & QC	Defect prevention; statistical QC; process optimization

- **TMM level parts:** Maturity goals, Supporting maturity goals, **ATRs** (Activities, Tasks, Responsibilities).
- **TPI — improve test process (4 steps):** ① Determine area ② Evaluate current state (baseline) ③ Identify next state + means ④ Implement changes.

CH 4 — Software Quality Assurance

- **QA = correctness aspect, dealing with defects. 3 generic categories** (mnemonic "**Prevent, Reduce, Contain**"):

Category	What it does	Example
Defect Prevention	Stop faults being injected	default values, input options
Defect Reduction	Detect & remove injected faults	inspection, testing
Defect Containment	Contain failures / limit damage	auto-save , exception handling

- **Defect prevention — 2 ways: Error source removal** (education/training — people-based) · **Error blocking** (block wrong actions, e.g. can't divide by 0, data validation). Also process improvement / formal methods.
- **Defect reduction techniques: Inspection** (static — informal walkthroughs vs formal inspections) · **Testing** · Boundary value analysis · Control/data flow analysis · Simulation/Prototyping.
- **Defect containment — 2 ways: Fault tolerance** (Recovery = rollback & redo; **NVP** = N-Version Programming) · **Safety assurance**.
- **Safety definitions (memorize):**
- **Safety** = accident-free · **Accident** = failure with severe consequences · **Hazard** = pre-condition to an accident.
- **Safety Assurance** = hazard analysis, hazard elimination/reduction/control, damage control.
- **Process variations:** Waterfall · Iterative (QA in increments) · Spiral (QA + risk mgmt) · Agile (XP, TDD, pair programming) · Maintenance (defect handling).
- **DC → V&V mapping:** Unit = **Verification**; Beta/Operation = **Validation**; Acceptance = **both, more validation**.

TRAP: Prevention is **never 100% effective** → that's why reduction & containment exist. NVP & auto-save = **containment**, not prevention.

CH 5 — Software Quality Engineering (SQE)

- **SQE** = meet/exceed quality expectations via appropriate QA activities while **minimizing cost & risk** under constraints.
- **Generic testing process (3 phases): Pre-QA** (planning) → **In-QA** (execution) → **Post-QA** (measurement/assessment/improvement).
- **Pre-QA quality goals:** identify quality perspective/expectation · select direct quality measures (quantified) · assess expectations vs. cost.

Term	Level	One line
Test Plan	High-level doc	objectives, scope, approach, resources, schedule of testing
Test Case	Low-level doc	one input/action + expected result to check a feature
Test Suite	Macro-level	collection of test cases run until stopping criteria

- **Regression testing** = reuse test cases from earlier versions (**no new cases designed**).
- **Post-QA** does **not** mean "after QA finishes" — runs **parallel** to QA.
- **Testing team models:**
- **Vertical** = organized **around one product** (dedicated people, many tasks for that product).
- **Horizontal** = **one kind of testing** across many products.
- **Mixed** = both (large organizations).

CH 6 — Software Testing

- **Testing** = execute software + observe behaviour; if failure → locate & fix fault; else gain confidence.
- **7 Principles** — mnemonic "**Please Eat Every Delicious Pizza Carefully, Alright**":

#	Principle	One line
1	Presence not absence	shows defects exist , never proves none
2	Exhaustive impossible	use risk/time/cost/priority
3	Early testing	earlier = cheaper to fix
4	Defect clustering	few modules hold most defects
5	Pesticide paradox	repeated tests stop finding new bugs → revise
6	Context dependent	safety-critical ≠ e-commerce
7	Absence-of-errors fallacy	no bugs ≠ usable/useful product

- **4 Testing levels + who:** Unit → Developer · Integration → Tester · System → Tester · Acceptance → End-user.
- **Regression testing:** selects/prioritizes/executes existing tests — **designs none**.
- **Developer** = constructive, builds, driven by delivery · **Tester** = destructive, finds defects, driven by quality.

	White-box (WBT)	Black-box (BBT)
Basis	Implementation / structural	Specification / functional
Sees	source code (control flow, data flow)	external interface only
Objects	small units	large systems
Timeline	early (unit/component)	late (system/acceptance)
Tester	developers	professional / 3rd-party (IV&V)

- **Gray-box** = mixed (middle levels, e.g. procedures in modules).
- **When to stop (Exit criteria):** *not* "no more defects found". **Resource-based** (time/money — irresponsible) vs **Quality-based** (goals reached) vs Activity completion.
- **Manual** = oldest, rigorous, hard-to-repeat, costly, slow · **Automated** = fast, repeatable, reliable, reusable, programmable.
- **Automate:** every-build/regression, data-driven, internal-detail (GUI), stress/load.
- **DON'T automate:** usability, logical errors, specs/design docs, one-time/ASAP, ad-hoc, tests without predictable results. **Automation cannot replace manual testing.**